

Plum Health Insurance

AI Claims Processing System

Component Contracts

This document defines the interface for every significant component in the system. Each contract specifies what the component accepts as input, what it produces as output, and what errors it can raise. These contracts are precise enough that another engineer could reimplement any single component without reading its source code.

1. Component Index

#	Component	File	Type
1	Document Verifier	agents/verifier.py	Agent
2	Document Parser	agents/parser.py	Agent (LLM)
3	Policy Evaluator	agents/evaluator.py	Agent
4	Fraud Detector	agents/fraud.py	Agent
5	Decision Synthesizer	agents/synthesizer.py	Agent

#	Component	File	Type
6	LangGraph Pipeline	agents/pipeline.py	Orchestrator
7	Policy Engine	core/policy_engine.py	Core (pure functions)
8	File Handler	core/file_handler.py	Core (file processing)
9	Claims Upload Endpoint	routers/upload.py	API
10	Claims JSON Endpoint	routers/claims.py	API

2. Agent 1 - Document Verifier

Runs before any LLM or policy checks. If it finds a problem it returns immediately. The pipeline stops and returns error messages to the caller - no further agents run.

Function Signature

```
run_document_verifier(submission: ClaimSubmission)
    -> tuple[list[DocumentVerificationError], TraceStep]
```

Input

Field	Type	INPUT - ClaimSubmission fields used
<code>member_id</code>	<code>str</code>	Used to validate patient name consistency across documents
<code>claim_category</code>	<code>ClaimCategory</code>	Determines which document types are required (from <code>policy_terms.json</code>)
<code>documents</code>	<code>list[DocumentInfo]</code>	Each document's <code>actual_type</code> , <code>quality</code> , <code>patient_name_on_doc</code> , <code>file_id</code> , <code>file_name</code>

Output

Field	Type	OUTPUT - tuple[errors, trace]
<code>errors</code>	<code>list[DocumentVerificationError]</code>	Empty list if all documents valid. Non-empty triggers early exit.
<code>trace</code>	<code>TraceStep</code>	<code>agent='document_verifier'</code> , <code>status=SUCCESS</code> or <code>FAILED</code> , <code>detail</code> describes what was checked

DocumentVerificationError Fields

Field	Type	ERROR OBJECT
<code>error_code</code>	<code>str</code>	WRONG_DOCUMENT_TYPE UNREADABLE_DOCUMENT PATIENT_MISMATCH
<code>message</code>	<code>str</code>	Human-readable. Names exactly what was uploaded and what is needed. Never generic.
<code>file_id</code>	<code>str</code> <code>None</code>	Which file triggered the error
<code>uploaded_type</code>	<code>DocumentType</code> <code>None</code>	What the system received

Field	Type	ERROR OBJECT
<code>required_types</code>	<code>list[DocumentType]</code> <code> None</code>	What was expected for this claim category
<code>details</code>	<code>dict</code> <code> None</code>	Extra data e.g. {names_found: {F001: 'rajesh', F002: 'arjun'}}

Checks Performed

Check	Triggers On	Error Code	Test Case
Required doc types present	Uploaded types don't match required types for claim_category	WRONG_DOCUMENT_TYPE	TC001
Document readability	doc.quality == UNREADABLE	UNREADABLE_DOCUMENT	TC002
Patient name consistency (pre-parse)	2+ distinct names found in patient_name_on_doc fields	PATIENT_MISMATCH	TC003
Patient name consistency (post-parse)	2+ distinct names extracted by GPT-4o from real uploaded files	PATIENT_MISMATCH	TC003

Errors Raised

This function never raises exceptions. All failures are returned as DocumentVerificationError objects in the list.

3. Agent 2 - Document Parser

Extracts structured data from uploaded documents. For test cases with content dicts, reads directly. For real file uploads with file_bytes, calls GPT-4o vision. Returns a ParsedClaim object consumed by all downstream agents.

Function Signature

```
run_document_parser(submission: ClaimSubmission, simulate_failure: bool = False)
    -> tuple[ParsedClaim, TraceStep]
```

Input

Field	Type	INPUT - ClaimSubmission fields used
documents[].content	dict None	Pre-structured data (test cases). If present, used directly without LLM.
documents[].file_bytes	bytes None	Raw file bytes (real uploads). Triggers GPT-4o vision call.
documents[].actual_type	DocumentType	Controls which extraction prompt is used for each document
documents[].quality	DocumentQuality	UNREADABLE documents are skipped, confidence reduced
claim_category	ClaimCategory	Passed to extraction prompt for context
simulate_failure	bool	If True, returns partial result with confidence=0.4 (TC011)

Output - ParsedClaim

Field	Type	OUTPUT - ParsedClaim fields
patient_name	str None	First non-null patient name found across all documents
diagnosis	str None	Primary diagnosis from prescription
treatment	str None	Treatment description if diagnosis not found
doctor_name	str None	Doctor's name with title
doctor_registration	str None	State medical registration number
hospital_name	str None	Hospital or clinic name from bill
treatment_date	str None	YYYY-MM-DD format
total_amount	float None	Sum of all bill totals across documents
line_items	list[dict]	List of {description, amount} from bills

Field	Type	OUTPUT - ParsedClaim fields
<code>medicines</code>	<code>list[str]</code>	Medicine names from prescription
<code>tests_ordered</code>	<code>list[str]</code>	Diagnostic tests from prescription
<code>overall_confidence</code>	<code>float</code>	0.0-1.0. Average confidence across all parsed documents.
<code>warnings</code>	<code>list[str]</code>	Low-confidence fields, skipped documents, extraction failures

GPT-4o Vision Call - Real Uploads

When `file_bytes` is present and `content` is `None`:

- `file_handler.process_file()` converts PDF to JPEG pages via `pymupdf`
- `build_vision_content()` encodes pages as base64 `image_url` messages
- GPT-4o is called with `response_format={type: json_object}` and `temperature=0`
- Response parsed for confidence float and warnings list in addition to fields
- On any exception: returns `ParsedDocument` with `confidence=0.2` and warning message

Errors Raised

This function never raises exceptions. LLM failures are caught and returned as warnings with `confidence=0.2`. The pipeline continues.

4. Agent 3 - Policy Evaluator

Applies every policy rule from policy_terms.json against the parsed claim. No LLM calls. All checks are deterministic pure Python functions from policy_engine.py.

Function Signature

```
run_policy_evaluator(submission: ClaimSubmission, parsed: ParsedClaim,
                     simulate_failure: bool = False)
    -> tuple[EvaluationResult, TraceStep]
```

Input

Field	Type	INPUT fields used
submission.member_id	str	Looked up in policy_terms.json members array
submission.claim_category	ClaimCategory	Controls which checks apply (e.g. pre-auth only for DIAGNOSTIC)
submission.treatment_date	str	YYYY-MM-DD used for waiting period calculation
submission.claimed_amount	float	Checked against per_claim_limit and sub-limits
submission.hospital_name	str None	Checked against network_hospitals for discount
parsed.diagnosis	str None	Used for waiting period, exclusion, and pre-auth checks
parsed.tests_ordered	list[str]	Checked for MRI/CT pre-auth requirement
parsed.line_items	list[dict]	Each item checked individually for exclusions (TC006)

Checks - In Order

Step	Check	Policy Source	Sets passed=False If
1	Member lookup	members[]	member_id not found in roster
2	Waiting period	waiting_periods{}	treatment_date within condition-specific wait from join_date
3	Pre-authorization	pre_auth{}	DIAGNOSTIC claim with MRI/CT above threshold, no pre-auth
4	Per-claim limit	coverage.per_claim_limit	claimed_amount exceeds limit (skipped for DENTAL/VISION)
5	Claim exclusions	exclusions[]	diagnosis or treatment matches exclusion keywords

Step	Check	Policy Source	Sets passed=False If
5b	Line-item exclusions	exclusions[]	Individual bill items match exclusion keywords (produces PARTIAL)
6	Amount calculation	copay%, network%	Always runs if passed=True. Applies network discount then copay.

Output - EvaluationResult

Field	Type	OUTPUT - EvaluationResult fields
<code>passed</code>	<code>bool</code>	True if all claim-level checks passed. False triggers REJECTED.
<code>rejection_reasons</code>	<code>list[RejectionReason]</code>	Enum values: WAITING_PERIOD PRE_AUTH_MISSING PER_CLAIM_EXCEEDED EXCLUDED_CONDITION DOCUMENT_MISMATCH
<code>rejection_messages</code>	<code>list[str]</code>	Human-readable explanation for each rejection. States eligible dates, limits, rule names.
<code>approved_amount</code>	<code>float</code>	0.0 if rejected. Post-discount and post-copay amount if passed.
<code>line_item_decisions</code>	<code>list[LineItemDecision]</code>	Per-item result when bill has multiple line items. Drives PARTIAL decision.
<code>calculation_breakdown</code>	<code>dict</code>	claimed, network_discount, after_network_discount, copay, approved_amount
<code>checks_run</code>	<code>list[str]</code>	Human-readable log of every check e.g. 'waiting_period: PASS'

Errors Raised

Never raises. If `simulate_failure=True`, returns `EvaluationResult` with `passed=True` and `confidence_delta=-0.3`. The pipeline continues.

5. Agent 4 - Fraud Detector

Checks the claim against fraud thresholds from policy_terms.json. Routes suspicious claims to MANUAL_REVIEW. Never auto-rejects - fraud signals are indicators, not proof.

Function Signature

```
run_fraud_detector(submission: ClaimSubmission, parsed: ParsedClaim,
                   simulate_failure: bool = False)
    -> tuple[FraudResult, TraceStep]
```

Input

Field	Type	INPUT fields used
submission.member_id	str	Passed to check_fraud_signals
submission.treatment_date	str	Used for same-day claim count
submission.claimed_amount	float	Checked against high_value_claim_threshold
submission.claims_history	list[ClaimsHistory]	Previous claims for this member. Each has date, amount, provider.

Checks

Signal	Threshold (from policy_terms.json)	Test Case
Same-day claim count	fraud_thresholds.same_day_claims_limit (default: 2)	TC009
High-value claim	fraud_thresholds.high_value_claim_threshold (default: 25000)	-

Output - FraudResult

Field	Type	OUTPUT - FraudResult fields
is_suspicious	bool	True if any signal was detected. Synthesizer routes to MANUAL_REVIEW.
signals	list[str]	Human-readable description of each signal found. e.g. '3 existing claims on 2024-10-30 (limit: 2)'
confidence_delta	float	-0.2 if suspicious, 0.0 if clean

Errors Raised

Never raises. If `simulate_failure=True`, returns `FraudResult` with `is_suspicious=False` and `confidence_delta=-0.1`.

6. Agent 5 - Decision Synthesizer

Combines the outputs of all previous agents into a single ClaimDecision. Applies decision priority order and calculates the final confidence score.

Function Signature

```
run_decision_synthesizer(submission, parsed, evaluation, fraud,  
                          all_traces, component_failures)  
-> ClaimDecision
```

Input

Field	Type	INPUT
<code>submission</code>	<code>ClaimSubmission</code>	Original submission for member_id and claimed_amount
<code>parsed</code>	<code>ParsedClaim</code>	overall_confidence used to reduce final score
<code>evaluation</code>	<code>EvaluationResult</code>	passed, rejection_reasons, approved_amount, line_item_decisions
<code>fraud</code>	<code>FraudResult</code>	is_suspicious, signals - fraud overrides all other decisions
<code>all_traces</code>	<code>list[TraceStep]</code>	Collected from all previous agents, appended to final decision
<code>component_failures</code>	<code>list[str]</code>	Names of any agents that threw exceptions during execution

Decision Priority

Priority	Condition	Decision	Test Case
1 (highest)	fraud.is_suspicious == True	MANUAL_REVIEW	TC009
2	evaluation.passed == False (no line items)	REJECTED	TC005, TC007, TC008, TC012
3	line_item_decisions has both approved and rejected items	PARTIAL	TC006
4	all line_item_decisions have approved_amount == 0	REJECTED	-
5 (default)	All checks passed	APPROVED	TC004, TC010, TC011

Confidence Score Calculation

Starts at 1.0. Reduced by:

- `parsed.overall_confidence` - 1.0 (parser quality, skipped for hard policy rejections)
- Sum of all agent `confidence_deltas` from traces
- `component_failures.length * 0.15`
- Clamped to range [0.1, 1.0]

Output - ClaimDecision

Field	Type	OUTPUT - ClaimDecision fields
<code>claim_id</code>	<code>str</code>	Random 8-character uppercase hex ID
<code>member_id</code>	<code>str</code>	Passed through from submission
<code>decision</code>	<code>DecisionType</code>	APPROVED PARTIAL REJECTED MANUAL_REVIEW
<code>approved_amount</code>	<code>float</code>	0.0 for REJECTED and MANUAL_REVIEW
<code>claimed_amount</code>	<code>float</code>	Original claimed amount from submission
<code>rejection_reasons</code>	<code>list[RejectionReason]</code>	Enum values from evaluation agent
<code>line_item_decisions</code>	<code>list[LineItemDecision]</code>	Per-item breakdown for PARTIAL decisions
<code>confidence_score</code>	<code>float</code>	0.0-1.0. See calculation above.
<code>message</code>	<code>str</code>	Human-readable explanation of the decision
<code>trace</code>	<code>list[TraceStep]</code>	Full audit trail from all agents including synthesizer
<code>component_failures</code>	<code>list[str]</code>	Any agents that failed during execution
<code>manual_review_recommended</code>	<code>bool</code>	True for MANUAL_REVIEW or when component failures occurred
<code>calculation_breakdown</code>	<code>dict None</code>	claimed, network_discount, copay, approved_amount

Errors Raised

Never raises. If synthesizer itself crashes, `_emergency_decision()` returns a `MANUAL_REVIEW` with `confidence=0.1`.

7. LangGraph Pipeline

Wires all 5 agents into a directed graph. Manages state between nodes. Catches exceptions at every node so the pipeline always completes.

Function Signature

```
run_pipeline(submission: ClaimSubmission) -> dict
```

Input

Field	Type	INPUT
<code>submission</code>	<code>ClaimSubmission</code>	Full claim submission including documents with optional <code>file_bytes</code>
<code>submission.simulate_component_failure</code>	<code>bool</code>	If True, document parser runs in failure mode (TC011)

Graph Structure

Node	Agent	Conditional Edge
verifier	Document Verifier	If errors found: route to END (returns errors). If clean: continue.
parser	Document Parser	Post-parse patient name cross-check runs here. Then continues to evaluator.
evaluator	Policy Evaluator	Always continues to fraud
fraud	Fraud Detector	Always continues to synthesizer
synthesizer	Decision Synthesizer	Routes to END

Output

Field	Type	OUTPUT - dict
<code>status</code>	<code>str</code>	'document_verification_failed' or 'completed'
<code>decision</code>	<code>str</code>	APPROVED PARTIAL REJECTED MANUAL_REVIEW (if status=completed)
<code>approved_amount</code>	<code>float</code>	If status=completed
<code>claimed_amount</code>	<code>float</code>	If status=completed
<code>confidence_score</code>	<code>float</code>	If status=completed

Field	Type	OUTPUT - dict
message	str	If status=completed
rejection_reasons	list[str]	If status=completed
line_item_decisions	list[dict]	If status=completed
calculation_breakdown	dict None	If status=completed
component_failures	list[str]	If status=completed
manual_review_recommended	bool	If status=completed
trace	list[dict]	Always present. Full audit trail.
errors	list[dict]	If status=document_verification_failed

Errors Raised

run_pipeline() never raises. Even if every agent fails, it returns a dict with status=completed and decision=MANUAL_REVIEW.

8. Policy Engine

Stateless pure functions that read from policy_terms.json. No LLM. No side effects. Each function can be called independently. All values come from the JSON file - nothing is hardcoded.

Functions

Function	Input	Output	Raises
get_member(member_id)	str	dict None - member record	Never
check_waiting_period(member_id, treatment_date, diagnosis_text)	str, str, str	tuple[bool, str None, date None] - (eligible, message, eligible_from)	Never
requires_pre_auth(claim_category, diagnosis_text, amount)	ClaimCategory, str, float	tuple[bool, str None] - (required, reason)	Never
is_excluded(description)	str	tuple[bool, str None] - (excluded, reason)	Never
check_per_claim_limit(amount)	float	tuple[bool, str None] - (within_limit, message)	Never
is_network_hospital(hospital_name)	str None	bool	Never
calculate_approved_amount(category, amount, hospital_name)	ClaimCategory, float, str None	dict with claimed, network_discount, copay, approved_amount	Never
check_fraud_signals(member_id, date, amount, history)	str, str, float, list[dict]	tuple[bool, list[str]] - (suspicious, signals)	Never
get_required_documents(category)	ClaimCategory	list[DocumentType]	Never

Amount Calculation Order

calculate_approved_amount applies discounts in this exact order:

- 1. Apply network discount on full claimed amount (if hospital is in network_hospitals list)
- 2. Apply co-pay percentage on post-discount amount
- 3. Sub-limit is informational only - YTD tracking is handled separately

TC010 example: ₹4,500 claimed at Apollo. 20% network discount = ₹900 off. ₹3,600 remains. 10% copay = ₹360 off. **Approved: ₹3,240.**

9. File Handler

Converts uploaded files into base64-encoded images for GPT-4o vision. PDFs are rendered to JPEG pages via pymupdf. Images are quality-checked via blur detection.

Main Function

```
process_file(file_bytes: bytes, file_id: str, file_name: str) -> FileContent
```

File Type Detection

File Type	Detection Method	Processing
PDF	Magic bytes: starts with %PDF	pymupdf renders each page to JPEG at 2x scale
JPEG	Magic bytes: starts with 0xFF 0xD8	Pillow opens, converts to RGB if needed, resize if > 2048px
PNG	Magic bytes: starts with 0x89P	Same as JPEG
Unknown	Extension fallback, then UNKNOWN	Returns FileContent with error field set

Output - FileContent

Field	Type	OUTPUT - FileContent
file_id	str	Passed through from input
file_name	str	Passed through from input
file_type	FileType	PDF IMAGE UNKNOWN
pages	list[PageContent]	One entry per page (PDFs) or one entry (images)
error	str None	Set if file could not be processed. pages will be empty.
is_readable	bool (property)	True if no error and at least one page exists
quality_label	str (property)	GOOD (>=5000) DEGRADED (>=500) UNREADABLE (<500) - Laplacian variance. PDFs always GOOD.

PageContent

Field	Type	PageContent fields
page_number	int	1-indexed page number
base64_data	str	Base64-encoded JPEG image data

Field	Type	PageContent fields
<code>media_type</code>	<code>str</code>	Always 'image/jpeg' (PDFs converted to JPEG by pymupdf)
<code>blur_score</code>	<code>float</code>	Laplacian variance score. Higher = sharper. PDFs always get 9999. Images: sharp=5000+, degraded=500-5000, unreadable<500.
<code>is_degraded</code>	<code>bool</code>	True if <code>blur_score</code> < 5000. Causes quality warning added to GPT-4o prompt.

build_vision_content

```
build_vision_content(file_content: FileContent, prompt: str) -> list[dict]
```

Builds the messages content list for a GPT-4o vision API call. Returns a list starting with the text prompt, followed by one `image_url` entry per page. Degraded pages get an additional text warning before their image entry.

Errors Raised

`process_file()` never raises. All errors are returned in `FileContent.error`. `build_vision_content()` never raises.

10. API Endpoints

POST /api/v1/claims/upload

Accepts real file uploads. Used by the React frontend.

Request - multipart/form-data

Field	Type	FORM FIELDS
<code>member_id</code>	<code>str (required)</code>	Member identifier e.g. EMP001
<code>policy_id</code>	<code>str (required)</code>	Policy identifier e.g. PLUM_GHI_2024
<code>claim_category</code>	<code>str (required)</code>	CONSULTATION DIAGNOSTIC PHARMACY DENTAL VISION ALTERNATIVE_MEDICINE
<code>treatment_date</code>	<code>str (required)</code>	YYYY-MM-DD format
<code>claimed_amount</code>	<code>float (required)</code>	Total amount being claimed
<code>hospital_name</code>	<code>str (optional)</code>	Hospital or clinic name. Checked against network list.
<code>ytd_claims_amount</code>	<code>float (optional, default 0)</code>	Year-to-date claims for this member
<code>document_types</code>	<code>str (optional)</code>	JSON array of DocumentType values matching files order e.g. ["PRESCRIPTION","HOSPITAL_BILL"]
<code>files</code>	<code>list[UploadFile]</code>	One or more PDF or image files

Response - 200 OK

Field	Type	SUCCESS RESPONSE
<code>status</code>	<code>str</code>	'completed'
<code>claim_id</code>	<code>str</code>	8-character claim reference
<code>decision</code>	<code>str</code>	APPROVED PARTIAL REJECTED MANUAL_REVIEW
<code>approved_amount</code>	<code>float</code>	Amount approved after discounts and copay
<code>confidence_score</code>	<code>float</code>	0.0-1.0
<code>message</code>	<code>str</code>	Human-readable decision explanation
<code>trace</code>	<code>list[dict]</code>	Full audit trail - one entry per agent

Response - 422 Unprocessable Entity

Error Code	When Raised	What to Do
<code>document_verification_failed</code>	Wrong doc type, unreadable doc, or patient mismatch	Return errors[] to caller. Do not resubmit

Error Code	When Raised	What to Do
		without fixing documents.
<code>invalid_claim_data</code>	<code>claim_category</code> value not in enum, malformed date	Fix the form field value and resubmit
<code>No files uploaded</code>	<code>files list</code> is empty	Include at least one file in the upload

POST /api/v1/claims

Accepts pre-structured JSON. Used for programmatic testing and the test suite.

Request - application/json

Accepts a full ClaimSubmission JSON object. Documents include content dicts instead of file_bytes. See models.py for full schema.

```
{"member_id": "EMP001", "policy_id": "PLUM_GHI_2024",
 "claim_category": "CONSULTATION", "treatment_date": "2024-11-01",
 "claimed_amount": 1500, "documents": [{"file_id": "F001",
 "actual_type": "PRESCRIPTION", "quality": "GOOD",
 "content": {"patient_name": "Rajesh Kumar", "diagnosis": "Viral Fever"}}]}
```

Response

Same response structure as /claims/upload.